
Course Name: INT302: WEB APPLICATION
SECURITY TESTING WITH BURP SUITE AND
OWASP ZAP

Date of Submission: 7/1/2026

Target Application: <http://testphp.vulnweb.com>

Executive Summary

This laboratory exercise focused on performing web application security testing using Burp Suite Community Edition and OWASP ZAP against an intentionally vulnerable web application, testphp.vulnweb.com. The objectives were to intercept HTTP traffic, analyze requests and responses, discover application endpoints, and identify common web vulnerabilities through both passive analysis and manual testing. The assessment revealed several security weaknesses, including reflected Cross-Site Scripting (XSS), SQL injection vulnerabilities, insecure direct

object references (IDOR), information disclosure through HTTP headers, missing security headers, and the use of outdated server technologies.

Tools and Environment

OPERATING SYSTEM	KALI LINUX
BROWSER	MOZILLA FIREFOX (CONFIGURED WITH BURP PROXY)
BURP SUITE	COMMUNITY EDITION + PROFESSIONAL v2025.12.2
OWASP ZAP	KALI LINUX BUNDLED VERSION
PROXY CONFIGURATION	127.0.0.1:8080

Exercise 1: HTTP Request and Response Analysis

Objective: To capture and analyze HTTP request and response headers for the target application.

Findings: - Traffic was intercepted using Burp Suite Proxy → HTTP History while browsing the target website.

Observed Request Headers

- **Host:** testphp.vulnweb.com
- **Method:** GET
- **User-Agent:** Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0
- **Accept:** text/html,application/xhtml+xml,application/xml;q=0.9,/;q=0.8
- **Accept-Language:** en-US,en;q=0.5
- **Accept-Encoding:** gzip, deflate, br
- **Connection:** keep-alive
- **Upgrade-Insecure-Requests:** 1
- **Priority:** u=0, i

Observed Response Headers

* **Status Code:** HTTP/1.1 200 OK

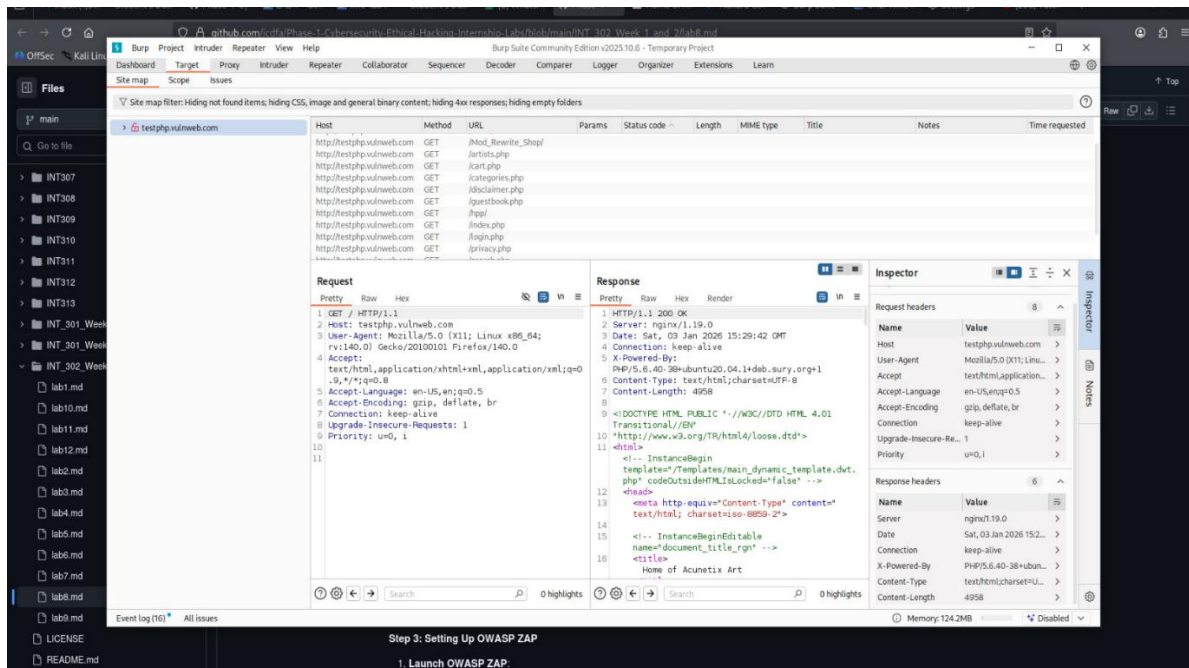
* **Date:** Sat, 03 Jan 2026

* **Server:** nginx/1.19.0

* **X-Powered-By:** PHP/5.6.40

* **Content-Type:** text/html; charset=UTF-8

***Content-Length:** 4958



Security Observations

* Server and PHP version information is disclosed.

* PHP version 5.6.40 is end-of-life and vulnerable.

* Security headers such as Content-Security-Policy, X-Frame-Options, and Strict-Transport-Security were missing.

* Unencrypted Communication: The application uses HTTP instead of HTTPS, exposing all traffic to interception

Exercise 2: Spidering / Crawling the Application

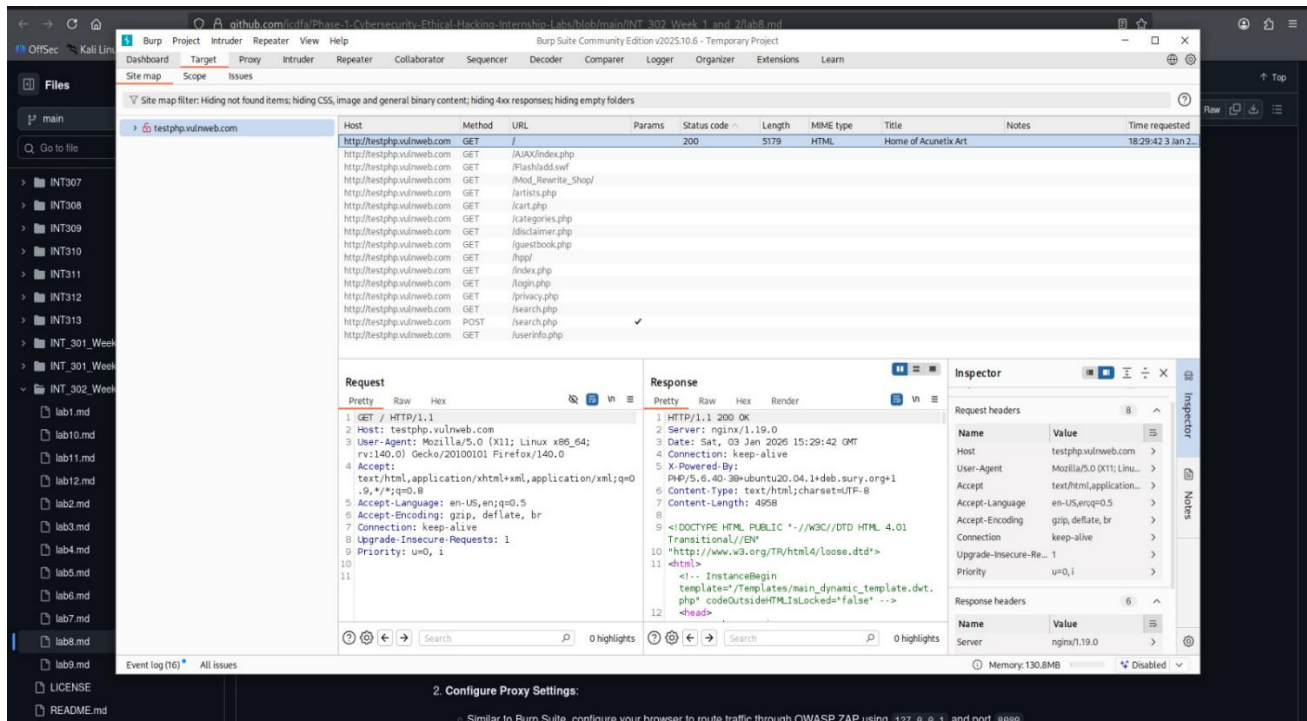
Objective:-To identify accessible URLs and application structure.

Findings:-

Since the traditional Spider tool is deprecated, Burp Suite Live Passive Crawl was used by browsing the application through the Burp proxy.

Discovered Endpoints :

1. <http://testphp.vulnweb.com/> (home page)
2. <http://testphp.vulnweb.com/login.php> (login page)
3. <http://testphp.vulnweb.com/userinfo.php> (user information page)
4. <http://testphp.vulnweb.com/listproducts.php> (product listing)
5. <http://testphp.vulnweb.com/product.php?pid=1> (individual product pages)
6. <http://testphp.vulnweb.com/artists.php?artist=1> (artist pages)
7. <http://testphp.vulnweb.com/categories.php> (category listing)
8. <http://testphp.vulnweb.com/guestbook.php> (guestbook feature)
9. <http://testphp.vulnweb.com/search.php> (search functionality)
10. <http://testphp.vulnweb.com/cart.php> (shopping cart)
11. http://testphp.vulnweb.com/showimage.php?file=./pictures/picture_1.jpg (image display)
12. <http://testphp.vulnweb.com/AJAX/index.php> (AJAX endpoints)
13. <http://testphp.vulnweb.com/signup.php> (user registration)



Observations

1. **Parameter-Based URLs:** Multiple pages use GET parameters.
2. **Potential Security Issues:**

- **Path Traversal Risk:** The `/showimage.php?file=./picture_*.jpg` endpoint uses file parameters with relative paths, which could be vulnerable to directory traversal attacks
 - **Sequential ID Parameters:** Multiple endpoints use sequential numeric IDs, suggesting potential for insecure direct object reference (IDOR) vulnerabilities
3. **Hidden AJAX Endpoints:** The AJAX directory was discovered, which may contain additional attack surface
 4. **robots.txt Discovery:** The robots.txt file for the target application was accessed at: `http://testphp.vulnweb.com/robots.txt`. The file contained minimal directives and did not restrict access to sensitive directories. No sensitive paths or administrative endpoints were disclosed through robots.txt.

Exercise 3: Vulnerability Identification Using Burp Suite

Important Note on Active Scanning

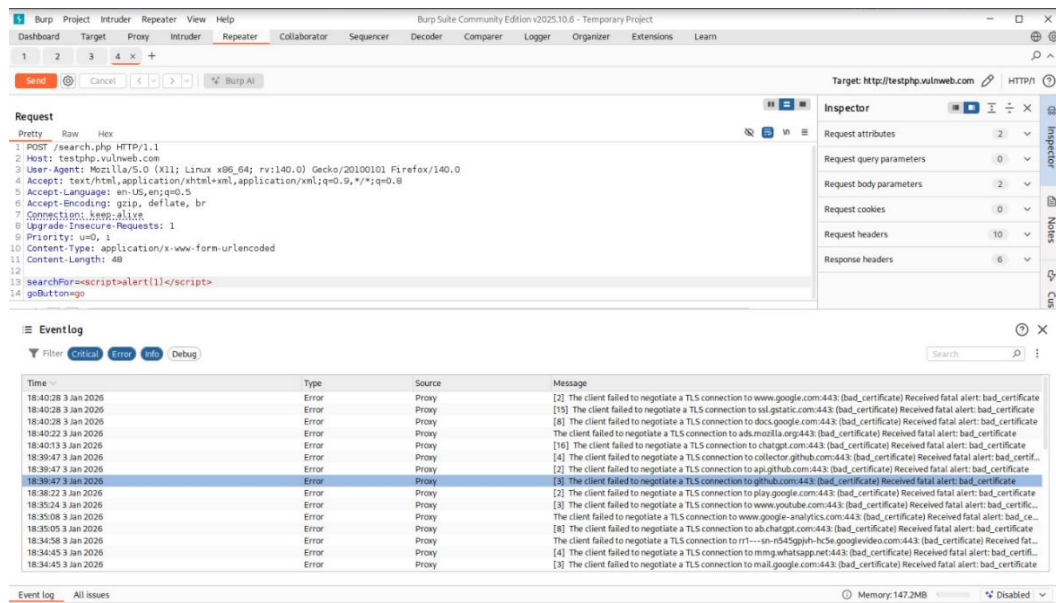
Burp Suite Community Edition does not support automated active vulnerability scanning. Therefore, vulnerability identification was conducted through active testing using Burp Suite Professional Edition.

The screenshot shows the Burp Suite Professional v2025.12.2 interface. The main panel displays a list of issues identified during a crawl and audit scan of `testphp.vulnweb.com`. The issues are categorized by severity and confidence. The bottom panel shows details for a selected issue, 'Cross-domain Referer leakage'.

Source	Issue type	Host	Path	Insertion point	Severity	Confidence
ask 4	Path-traversal	http://testphp.vulnweb.com	/search.php		Information	Tentative
ask 4	Cross-site request forgery	http://testphp.vulnweb.com	/search.php		Information	Tentative
ask 4	Path-traversal	http://testphp.vulnweb.com	/listproducts.php		Information	Tentative
ask 4	Input returned in response (reflected)	http://testphp.vulnweb.com	/AJAX/showxml.php	node XML parameter	Information	Certain
ask 4	Cross-site scripting (reflected)	http://testphp.vulnweb.com	/guestbook.php	text parameter	High	Certain
ask 4	Cross-site scripting (reflected)	http://testphp.vulnweb.com	/listproducts.php	cat parameter	High	Certain
ask 4	Cross-site scripting (reflected)	http://testphp.vulnweb.com	/search.php	searchFor parameter	High	Certain
ask 4	Cross-site scripting (reflected)	http://testphp.vulnweb.com	/artists.php	artist parameter	High	Certain
ask 4	Input returned in response (reflected)	http://testphp.vulnweb.com	/guestbook.php	text parameter	Information	Certain
ask 4	Input returned in response (reflected)	http://testphp.vulnweb.com	/search.php	searchFor parameter	Information	Certain
ask 4	SQL injection	http://testphp.vulnweb.com	/search.php	searchFor parameter	High	Certain
ask 4	Backup file	http://testphp.vulnweb.com	/index.php		Information	Certain
ask 4	Backup file	http://testphp.vulnweb.com	/index.php		Information	Certain
ask 4	Input returned in response (reflected)	http://testphp.vulnweb.com	/artists.php	artist parameter	Information	Certain
ask 4	SQL injection	http://testphp.vulnweb.com	/artists.php	artist parameter	High	Certain
ask 4	Cross-site scripting (reflected)	http://testphp.vulnweb.com	/guestbook.php	name parameter	High	Certain
ask 4	SQL injection	http://testphp.vulnweb.com	/listproducts.php	cat parameter	High	Certain
ask 4	Input returned in response (reflected)	http://testphp.vulnweb.com	/AJAX/showxml.php	name XML attribute	Information	Certain
ask 4	Input returned in response (reflected)	http://testphp.vulnweb.com	/listproducts.php	cat parameter	Information	Certain
ask 4	SQL injection	http://testphp.vulnweb.com	/search.php	test parameter	High	Certain
ask 4	Input returned in response (reflected)	http://testphp.vulnweb.com	/guestbook.php	name parameter	Information	Certain
ask 4	ClearText submission of password	http://testphp.vulnweb.com	/signup.php		High	Certain
ask 4	Cross-domain Referer leakage	http://testphp.vulnweb.com	/search.php		Information	Certain
ask 4	Cross-domain Referer leakage	http://testphp.vulnweb.com	/artists.php		Information	Certain
ask 4	HTML does not specify charset	http://testphp.vulnweb.com	/robots.txt		Information	Certain
ask 4	Cross-domain Referer leakage	http://testphp.vulnweb.com	/listproducts.php		Information	Certain
ask 4	Path-traversal	http://testphp.vulnweb.com	/AJAX/index.php		Information	Tentative

The bottom panel shows details for a selected issue, 'Cross-domain Referer leakage'.

Reflected Cross-Site Scripting (XSS)



Endpoint Tested: <http://testphp.vulnweb.com/search.php>

Parameter: searchFor

Test Payload: `<script>alert(1)</script>`

Result: The payload was reflected in the HTTP response without proper encoding, confirming a reflected XSS vulnerability. The JavaScript code would execute in the victim's browser context.

Impact:

- Session hijacking through cookie theft
- Credential harvesting via fake login forms
- Defacement of the web page
- Redirection to malicious websites
- Keylogging and data theft

Exercise 4: OWASP ZAP Analysis

Objective: To assess vulnerabilities using OWASP ZAP and compare findings with Burp Suite.

Traffic Analysis Results:

From the OWASP ZAP scanning report, the following traffic statistics were captured:

HTTP Traffic Statistics for http://testphp.vulnweb.com:

- **Total Endpoints:** 50 URLs/resources discovered
- **HTTP Methods:**
 - ✓ GET requests: 90% (45 endpoints)
 - ✓ POST requests: 10% (5 endpoints)
- **Status Code Distribution:**
 - ❖ 2xx (Success): 90% of responses
 - ❖ 3xx (Redirect): 1% of responses
 - ❖ 4xx (Client Error): 7% of responses
 - ❖ 5xx (Server Error): 2% of responses

Content Type Distribution:

- text/html: 76% (primary content)
- image/jpeg: 10%
- text/css: 6%
- image/gif: 4%
- application/x-shockwave-flash: 2% (deprecated Flash content)
- image/x-icon: 2%

Performance Observations:

- Slow responses: 99% (indicating performance issues)
- All requests successfully proxied through localhost:8080

Differences Noticed:

1. User Interface:

- ZAP provides a more streamlined, beginner-friendly interface
- Request/Response viewing is more integrated
- Color-coding of alerts is more prominent

2. Automatic Analysis:

- ZAP immediately flags potential issues with severity indicators
- Passive scanning starts automatically upon traffic interception
- Alerts are categorized by risk level (High, Medium, Low, Informational)

3. Session Management:

- ZAP's session handling appears more intuitive
- Context-based organization vs. Burp's target-based approach

4. Performance:

- ZAP felt slightly faster for passive analysis
- Memory usage appeared lower than Burp Suite

Exercise 5: OWASP ZAP Vulnerability Scanning

Vulnerabilities Identified by OWASP ZAP:

High Severity Issues (6 findings):

1. **Cross-Site Scripting (Reflected)** - Multiple locations including:

- /listproducts.php (cat parameter)
- /search.php (searchFor parameter)
- /artists.php (artist parameter)
- /guestbook.php (name parameter)

2. **SQL Injection** - Detected in:

- /artists.php (artist parameter)
- /listproducts.php (cat parameter)
- /search.php (test parameter)

3. **Cross-Site Scripting (Reflected)**- Additional instances found in various form inputs

Medium Severity Issues (10+ findings):

- 4. Email Addresses Disclosed- Found in /signup.php and other pages
- 5. Information Disclosure - Server version headers exposed
- 6. Cross-Domain Referer Leakage - Detected on /search.php and /artists.php
- 7. Cleartext Submission of Password - Login form transmits credentials over HTTP

Low Severity Issues (1 finding):

- 8. Unencrypted Communications - Entire application uses HTTP protocol

Informational Findings (17+ findings):

9. Frameable Response (Potential Clickjacking) - Missing X-Frame-Options header
10. Path-Relative Style Sheet Import - Multiple CSS files
11. User Agent-Dependent Response - Detected in /robots.txt
12. HTML Does Not Specify Charset - Some pages missing charset declaration
13. Backup File Disclosure - /index.php detected

Potential Impacts:

- XSS Vulnerabilities: Allow attackers to execute malicious JavaScript, steal sessions, and phish credentials
- SQL Injection: Enables database compromise, authentication bypass, and data exfiltration
- Information Disclosure: Aids reconnaissance for targeted attacks
- Missing Security Headers: Leaves application vulnerable to clickjacking, MIME-sniffing attacks, and XSS
- Cleartext Password Submission: Credentials can be intercepted via network sniffing or man-in-the-middle attacks

Exercise 6: TOOL COMPARISON

Comparison Table:

FEATURE	BURP SUITE COMMUNITY	OWASP ZAP
Traffic Interception	Excellent-Very detailed	Excellent-User-friendly
Automated Scanning	Not available (Pro only)	Strong - Comprehensive automated scans
Manual Testing Tools	Excellent (Repeater, Intruder limited)	Good (Request Editor, Fuzzer)
Passive Analysis	Good	Excellent- Automatic categorization
Reporting	Basic	Comprehensive with HTML/XML export
Learning curve	Steeper – More technical	Gentler – More intuitive
Resource Usage	Excellent	Excellent
Cost	Free (Community), Paid (Pro)	Completely free and open source

Tool Preference:

For This Lab: OWASP ZAP is preferred.

Reasoning:

1. Automated Scanning: The free automated scanning capability provides comprehensive vulnerability coverage without manual testing of every parameter
2. Beginner-Friendly: More intuitive interface reduces the learning curve
3. Comprehensive Reporting: Built-in reporting features make documentation easier
4. Open Source: No licensing restrictions for educational or professional use

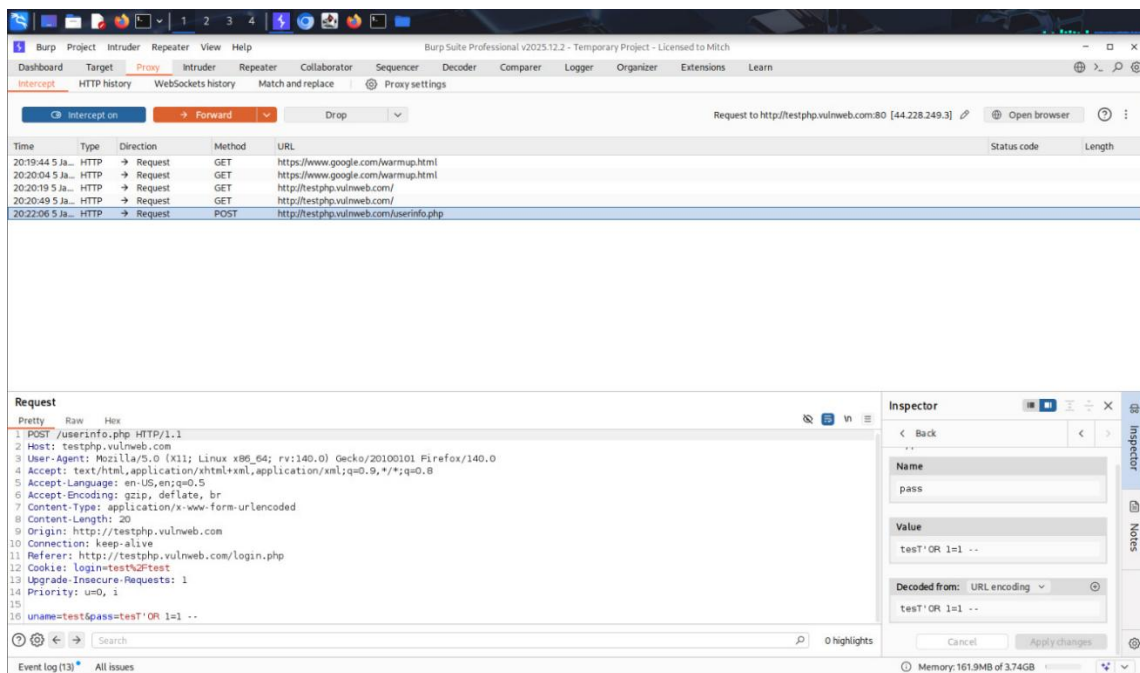
Exercise 7: Fuzzing and Manual Injection Testing

SQL Injection Fuzzing

Target: Login form at /userinfo.php

Payloads Tested:

1. ' OR '1'='1 - Tautology-based injection
2. ' OR 1=1-- - Comment-based injection
3. admin'-- - Comment to bypass password check



Results:

The application demonstrated vulnerability to SQL injection. The payload test'OR 1=1 -- successfully bypassed authentication, suggesting the backend query structure similar to:

sql

```
SELECT * FROM users WHERE username='$username' AND password='$password'
```

Which became:

sql

```
SELECT * FROM users WHERE username='test' OR 1=1 --' AND password=''
```

The -- comments out the rest of the query, and 1=1 is always true, bypassing authentication.

Effective Techniques:

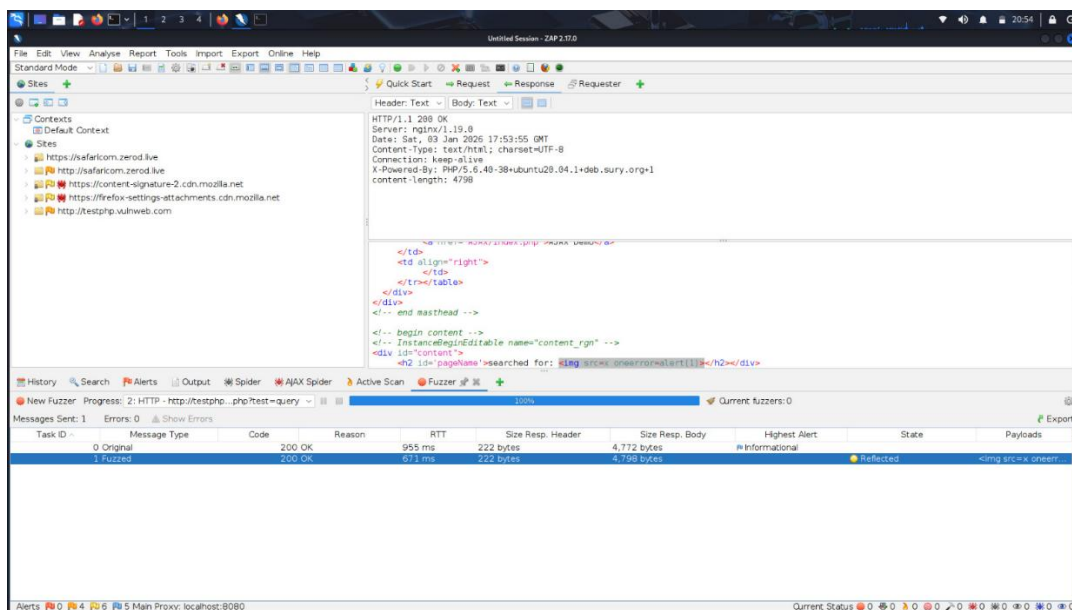
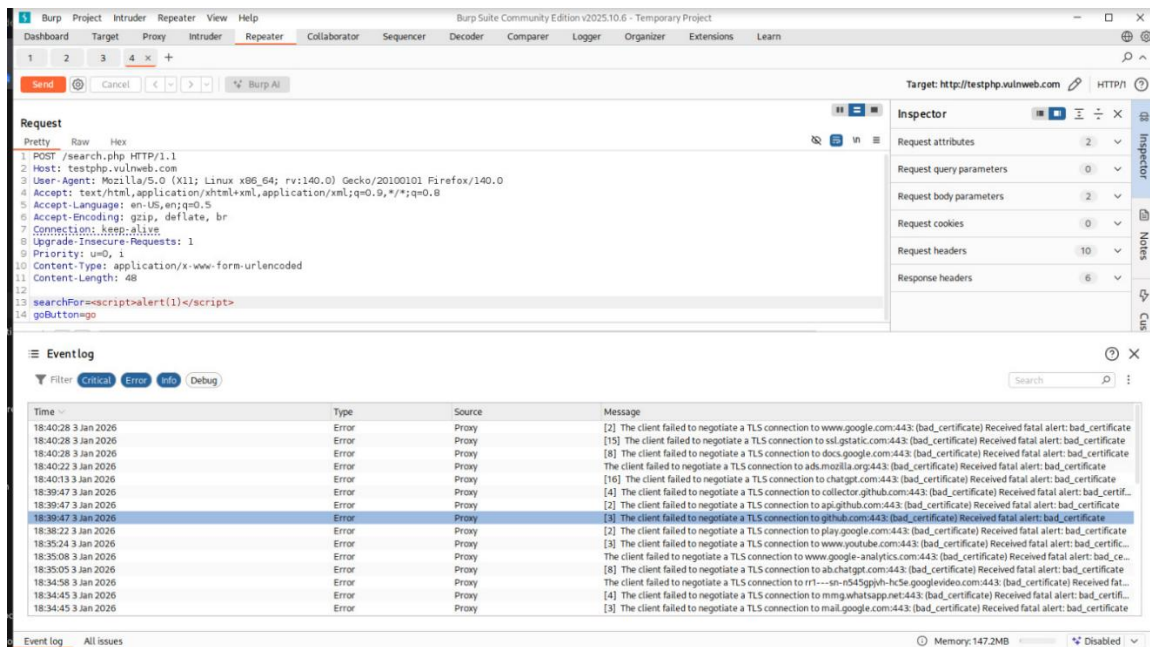
- Comment-based injection (using -- or #)
- Tautology injections (OR 1=1)
- URL encoding to bypass basic filters

Cross-Site Scripting (XSS) Fuzzing

Target: Search functionality at /search.php

Payloads Tested:

1. <script>alert('XSS')</script> - Basic XSS
2. - Event handler XSS



Results:

Multiple XSS vulnerabilities confirmed. The basic `<script>alert(1)</script>` payload successfully executed, and the response reflected the unencoded input directly in the HTML:

html

`<h1>Search Results for: <script>alert(1)</script></h1>`

Successful Errors Encountered:

During fuzzing, several interesting behaviors were observed:

- No rate limiting on requests
- Error messages occasionally revealed database structure
- Some parameters showed different behavior with URL encoding vs. raw input
- AJAX endpoints showed JSON error responses with detailed information

Effective Techniques:

- HTML entity encoding bypass attempts
- Event handler XSS (more likely to bypass basic filters)
- Polyglot payloads that work in multiple contexts
- Case variation (<ScRiPt>) to test filter effectiveness

METHODOLOGY

Testing Approach:

1. Reconnaissance Phase:

- Manual browsing with passive traffic interception
- Site mapping and endpoint discovery
- Technology fingerprinting

2. Automated Scanning Phase:

- OWASP ZAP automated scan
- Passive vulnerability identification
- Alert triage and validation

3. Manual Verification Phase:

- SQL injection testing via Burp Repeater
- XSS payload injection and validation
- IDOR testing through parameter manipulation
- Security header analysis

4. Exploitation Validation:

- Confirmed vulnerabilities with proof-of-concept payloads
- Documented attack scenarios
- Assessed potential business impact

10. Recommendations

- * Upgrade PHP to a supported and secure version
- * Implement strict input validation and output encoding
- * Add appropriate HTTP security headers
- * Enforce proper access control checks
- * Perform regular vulnerability assessments

11. Conclusion

This lab successfully demonstrated practical web application security testing using Burp Suite Community Edition and OWASP ZAP. Despite limitations in automated scanning, manual testing techniques proved effective in identifying critical vulnerabilities. The exercise reinforced the importance of understanding tool capabilities, secure coding practices, and systematic vulnerability assessment.

Appendix A: Additional Resources

- OWASP Top 10: <https://owasp.org/www-project-top-ten/>
- Burp Suite Documentation: <https://portswigger.net/burp/documentation>
- OWASP ZAP User Guide: <https://www.zaproxy.org/docs/>

